LIFEOS V34

Каноническое описание продукта

Personal System Operating Platform
Master Vision, Scope Ledger, Architecture and Build Plan

Дата
9 июля 2026

Формат
читабельный PDF

Цель
объяснить проект без потери scope

Версия для обсуждения с разработчиками, инвесторами, Cursor, Claude Code, Fable и будущими агентами.

Оглавление

Как читать этот документ

Сначала вывод, потом архитектура, потом детали и приложения.

Этот PDF перепаковывает большой технический markdown в нормальный продуктовый документ. Он не просто перечисляет функции. Он фиксирует механизм, через который LifeOS должен уметь создавать будущие функции и системы без переписывания ядра.

Что важно сразу

LifeOS нельзя понимать как заметки, чат, планер или dashboard. Это персональная операционная платформа.

Что защищает scope

System Factory, Artifact Ledger, Presentation Runtime, Adapter Runtime, Data Fabric, Model Fabric и Package Runtime.

Как пользоваться PDF

Для инвестора читать разделы 1, 2, 7, 21. Для разработчика - 3-20 и приложения. Для AI-агента - особенно 22-24.

Честное ограничение

Нельзя математически доказать, что в продукте, который должен позволять создавать любые будущие системы, перечислены вообще все будущие функции. Поэтому документ защищает не только список функций, а архитектурную формулу, через которую новая функция становится системой.

1. Самое короткое объяснение

LifeOS - это персональная операционная платформа для жизни, знаний, действий, данных, AI, автоматизаций и пользовательских систем.

Она должна заменить не одно приложение, а разрозненный стек человека: заметки, Obsidian, Notion, Telegram-потоки, планеры, базы, читалки, медиа, транскрибацию, граф знаний, AI-чаты, локальные LLM, workflow-автоматизации, smart-home панели, личные dashboard и marketplace систем.

Personal data → **Artifacts →** **Events →** **Channels →** **Renderers →** **Systems →**

Agents → **Adapters →** **Marketplace**

Главная идея

Пользователь не обязан ждать, пока разработчик вручную добавит каждую будущую функцию. LifeOS должен дать primitives, builder, adapters, packages, themes и AI-runtime, чтобы пользователь мог создавать нужные системы сам.

2. Чем LifeOS не является

Неправильное понимание	Почему это урезает проект
Приложение для заметок	Пропадают agents, workflows, marketplace, local AI, smart-home, system builder и artifact ledger.
Клон Notion или Obsidian	Пропадает private data OS, model fabric, receipts, action layer и adapter runtime.
AI chat	Чат не заменяет граф, поиск, feed, artifacts, data control, builder и permissions.
Dashboard	Dashboard показывает данные, но не создаёт системы, не управляет действиями и не хранит provenance.
Marketplace шаблонов	Marketplace должен быть extension layer поверх package runtime, а не витрина без kernel.
Умный дом	Smart-home - одна подсистема, не весь LifeOS.
Local LLM оболочка	Local LLM - один provider внутри Model Fabric.
Огромный монолит	Если одна подсистема падает, всё остальное должно продолжать работать.

3. Абсолютный продуктовый канон

1. Собирать личную информацию в одном приватном пространстве: заметки, файлы, книги, аудио, видео, транскрипты, задачи, источники, решения, receipts, граф связей, события устройств и AI-результаты.
2. Показывать происхождение результата: source, receipt, provenance, status, owner decision, rollback и export path.
3. Искать всё: текст, семантика, фильтры, теги, типы, источники, artifacts, systems, graph neighborhood.
4. Связывать всё в LifeGraph: знания, заметки, источники, задачи, книги, люди, проекты, устройства, события, агенты и артефакты.
5. Планировать и действовать через Today, Planning, Focus, Calendar и Weekly без скрытой мутации.
6. Создавать пользовательские системы через primitives: Entity, Field, Relation, View, Action, Trigger, Workflow, Agent, Artifact, Receipt, Policy, Budget, Dashboard, Channel, Package, Adapter, Renderer.
7. Настраивать внешний вид везде: один artifact может быть message bubble, card, table row, graph node, dashboard widget, terminal log, timeline event или reader highlight.

8. Подключать зрелые внешние движки через adapters/sidecars, а не переписывать всё с нуля.
9. Запускать cloud AI, свои API keys или локальные модели через BYOK, local LLM, Ollama/LM Studio/custom endpoints, privacy routing, budgets и model-call receipts.
10. Работать local-first: laptop, desktop, home-server, backup, export, restore, encrypted storage, secrets vault и cloud only by consent.
11. Давать screen/Jarvis-помощника только через permissioned screen context, local-first processing, proposal engine and receipts.
12. Подключать smart home через adapter к local smart-home engine with safety policies.
13. Иметь marketplace систем, templates, themes, renderers, adapters, workflows, agents and packages.
14. Расширяться без переписывания kernel: новая система устанавливается через manifest, permissions, data model, events, renderers, health and receipts.

4. Состояние текущего корпуса

Текущий корпус LifeOS уже содержит большой объём документов, реестров, audit-ledger материалов, маршрутов и исходного кода. Проблема не в отсутствии идей, а в том, что идеи размазаны по огромному массиву и должны быть собраны в один Scope Preservation Ledger.

Метрика	Значение
Text-like files/entries scanned	807
Approx chars	159,321,360
Lines	1,035,914
Token estimate low	36,209,400
Token estimate mid	44,255,933
Token estimate high	56,900,486

Theme bucket	Mentions	Files
system_factory	14,949	94
kernel_modularity	524	26
multi_agent	4,936	70
obsidian_graph_search	87,734	341
data_fabric	17,240	290
ai_hub_model_router	30,066	322
screen_jarvis	5,835	138
smart_home	9,155	141
marketplace_plugins	44,515	311
reader_media	37,766	289

Theme bucket	Mentions	Files
privacy_local_first	73,353	350
feed_internal_internet	16,179	279

Вывод

Темы не отсутствуют. Их надо нормализовать в архитектурные контракты, чтобы Cursor, Claude Code, Fable или человек не теряли scope между задачами.

5. Текущие 20 canonical modules v33

ID	Module
V33-M01	LifeGraph
V33-M02	Growth-first Personal Twin
V33-M03	AI Design Studio
V33-M04	Model Hub
V33-M05	Agent / Workflow Studio
V33-M06	Adaptive Home / Today
V33-M07	QC Contract Tests
V33-M08	Entity Resolution
V33-M09	Undo / Grace / Anti-shame
V33-M10	Consent-forward Privacy
V33-M11	Predictive Shortcuts / Inline Actions
V33-M12	Marketplace Extensions
V33-M13	Universal Builder / Custom Systems
V33-M14	Pack Economy
V33-M15	Trust Center
V33-M16	Local-first Memory/Search
V33-M17	Screen / Computer Companion
V33-M18	Media Transcript Engine
V33-M19	Source Sentinel / Research Hygiene
V33-M20	Release / Readiness Spine

В v34 эти модули не выбрасываются. Они поднимаются на уровень платформенной архитектуры: LifeGraph становится Graph Fabric, Model Hub становится Model Fabric, Universal Builder становится System Factory, Trust Center становится Data Control and Permission Control, Screen Companion становится permissioned Jarvis subsystem.

6. Главная архитектура LifeOS

LifeOS должен быть построен как kernel plus subsystem ecosystem. Kernel маленький, стабильный и живучий. Подсистемы устанавливаются, отключаются, индексируются, получают permissions и health state.

```
LifeOS Kernel
Identity and local profile
Permissions, capabilities and policies
Secrets vault
Module registry
Subsystem lifecycle manager
System Factory
Artifact Ledger
Event Bus
Channel and Stream Runtime
Presentation Runtime
Theme, Renderer and Layout Runtime
Data Fabric
Search Fabric
Graph and Relation Fabric
Model Fabric
Agent Runtime
Workflow Runtime
Connector and Adapter Runtime
Plugin and Package Runtime
Job Queue and Workers
Sync, Backup, Export and Restore
Health and Degraded Mode Layer
UI Shell
```

Kernel

Общие правила, права, события, данные, поиск, граф, model routing, artifacts, packages and health.

Subsystems

Notes, Obsidian bridge, reader, smart-home, Jarvis, marketplace, finance, travel, learning and user-created systems.

Adapters

Подключение зрелых внешних движков без переписывания их с нуля.

Presentation Runtime

Один artifact может иметь разные renderers, themes and layouts.

7. System Factory: система создания систем

Это сердце проекта. Если System Factory потерять, LifeOS станет набором страниц. Если сохранить, пользователь сможет создавать свои системы даже для функций, которые мы сегодня не назвали.

Главный закон

LifeOS Core не должен знать все будущие функции. LifeOS Core должен знать, как создавать, запускать, связывать, искать, отображать, защищать, отключать и переносить любые системы.

Primitive	Meaning
Entity	Любой объект: note, book, task, device, person, source, project, package.
Field	Свойство entity: title, status, date, file path, model provider, room.
Relation	Связь: note-source, highlight-book, device-room, agent-run.
View	Table, board, calendar, graph, feed, reader, dashboard, map.
Action	Import, summarize, approve, rollback, export, install, run.
Trigger	Time, new file, source imported, device event, agent completed.
Workflow	Цепочка actions, triggers and conditions.
Agent	AI-исполнитель с role, tools, budget, permissions and memory policy.
Artifact	Проверяемый результат: note, plan, receipt, import report, diff.
Receipt	Доказательство: who, what, when, source, model, status, rollback.
Permission	Capability: read notes, call cloud model, capture screen, control device.
Secret	API keys, tokens, provider credentials and local endpoints.
Adapter	Нормализатор внешнего engine в LifeOS contract.
Package	Устанавливаемая system, theme, renderer, workflow, agent or adapter.
Renderer	UI-компонент для artifact, entity or event.
Theme	Design tokens: colors, type, spacing, radius, density.
Channel	Поток событий: project feed, smart-home feed, agent feed.
Policy	Privacy, retention, cloud/local routing and safety confirmations.
Budget	Tokens, cost, time, device compute, disk and frequency.
Health State	Alive, degraded, failed, disabled, needs_config, permission_blocked.

8. Artifact, Event, Channel, Presentation Runtime

Telegram-like feed не должен быть каноном данных. Это только один renderer. Канон данных: source or action creates artifact; artifact emits event; event flows into channel; channel chooses renderer, theme and layout.

Source / Action / Agent / Adapter → | Artifact → | Event → | Channel / Stream →

Renderer → | Theme → | Layout → | User ViewPreset

Один artifact может выглядеть как	Где используется
Message bubble	Life Feed, Telegram-like stream
Card	Dashboard, project cockpit, reader hub
Table row	Database view, audit ledger, search results
Graph node	LifeGraph and knowledge graph
Timeline item	Daily log, activity history, agent run timeline
Terminal log	Developer/project mode
Reader highlight	Books and media subsystem
Smart-home alert	Device/event channel
Marketplace listing	Package discovery
Data-control receipt	Trust Center and audit surface

Смысл

Данные не дублируются ради дизайна. Меняется renderer, theme, density, grouping and layout. Это делает дизайн пользовательским слоем, а не хардкодом.

9. Дизайн, который пользователь настраивает везде

LifeOS должен иметь не просто светлую и тёмную тему, а Presentation System: design tokens, themes, layouts, renderers, view presets, component variants, per-system overrides and marketplace design packs.

Слой	Что делает
Design Tokens	Цвета, типографика, spacing, radius, shadows, density.
Themes	Единый визуальный стиль: minimal, dense, paper, terminal, glass, accessibility.
Renderers	Способ показать artifact или entity: bubble, card, table, timeline, graph, dashboard.
Layouts	Раскладки панелей и виджетов.
View Presets	Комбинация channel + renderer + theme + layout + density.
Marketplace Packs	Темы, renderer packs, reader skins, smart-home panels, dashboards.

Project Command Center

Renderer: terminal log. Theme: dense dark.
Grouping: by agent. Density: compact.

Reading Hub

Renderer: elegant cards. Theme: paper reader.
Grouping: by book. Density: comfortable.

Smart Home

Renderer: room cards. Theme: high contrast.
Grouping: by room and risk tier.

10. Adapter Runtime: не писать всё с нуля

LifeOS не должен переписывать годы работы зрелых open-source продуктов. Он должен подключать их как adapters, sidecars, embedded libraries or importers, а сверху давать unified artifact, event, permission, search, graph and presentation layer.

Domain	Candidate engines / references	Правильное использование
Quick capture / feed	Memos-like	Adapter or reference for capture and timeline.
PKM / Obsidian-like	SiYuan, Logseq-like, Obsidian bridge	Import/export/reference/adapter; license review required.
Rich editor	Tiptap-like	Headless editor inside notes/artifacts.
Visual builder	Craft.js, GrapesJS-like	Layout builder, dashboard builder, presentation builder.
Relational database builder	NocoDB, Directus-like	Data builder, schema editing, generated APIs or reference.
Smart home	Home Assistant-like	Adapter, not rewrite.
Visual AI agents	Flowise-like	Agent/workflow sidecar or reference.
Automation workflows	n8n-like	Adapter/reference; license review required.
Local models	Ollama, LM Studio, custom endpoint	Model provider adapters.

LifeOS добавляет поверх них

Artifact Ledger, receipts, Data Control, cross-system search/graph/feed, privacy policies, model routing, package runtime, renderer marketplace, health/degraded modes and unified UI shell.

11. Data Fabric: ткань данных

LifeOS не должен жить на одной плоской базе. Нужна логическая ткань данных, даже если первый MVP физически использует один SQL store плюс filesystem.

Слой	Зачем нужен
Relational Store	Профили, системы, packages, permissions, entities, jobs, providers, devices.
Document Store	Markdown, rich docs, source excerpts, agent reports, journals, specs.
Object/File Store	PDF, книги, аудио, видео, screenshots, attachments, exports, backups.
Event Log	История событий: import, artifact created, model call, package installed, device changed.
Artifact Store	Проверяемые результаты и receipts.
Graph Store	Nodes and edges: source-note-task-project-agent-device-package.
Full-text Search Index	Быстрый поиск по тексту, типам, системам, источникам and receipts.
Vector Index	Semantic search and RAG, only when privacy policy permits.
Secrets Vault	API keys, tokens, model endpoints, smart-home credentials.
Sync Queue	Offline-first, home-server sync, retries and conflict handling.
Backup/Export Bundles	Portability, restore, migration and trust.

12. Privacy, local-first, home-server

LifeOS должен быть пользовательским, а не SaaS-ловушкой. Данные человека должны быть переносимы, экспортируемы, удаляемы и по возможности локальны.

Local single-device

Локальная app, локальная база, локальные файлы, optional local models.

Desktop + daemon

UI общается с LifeOS background service.

Home server

Mini PC, NAS, Raspberry Pi or домашний сервер как личный узел.

Hybrid cloud

Cloud sync or cloud AI только с явным consent and receipts.

- No hidden cloud call.
- No hidden memory write.
- No hidden graph mutation.
- No hidden task or calendar mutation.
- No hidden screen capture.
- No hidden smart-home control.

- No hidden external share.
- No destructive action without preview, confirmation and receipt.
- Export, delete, rollback and backup must be visible.

13. Model Fabric: cloud, BYOK, local LLM

AI в LifeOS не равен одному чат-боту. Это routing fabric: cloud providers, user API keys, local models, home-server endpoints, budgets, privacy policies and receipts.

```
Model Fabric
Provider Registry
BYOK Secrets
Local LLM Connectors
Home Server Model Endpoints
Capability Registry
Task Router
Privacy Policies
Cost, Token and Device Budgets
Fallback Rules
Model Call Receipts
Per-system Permissions
```

Task type	Default routing
Private journal summary	Local model only; cloud forbidden by default.
Architecture review	Premium cloud or local large model; confirmation if cost threshold is crossed.
Screen assistant	Local vision first; cloud only by explicit consent; no screenshot storage by default.
Cheap tagging	Local small model first; cheap cloud fallback only if policy allows.

14. Agent Runtime и multi-agent orchestration

Multi-agent не должен стать роем бесконтрольных чатов. Каждый агент - управляемый worker with role, tools, permissions, memory policy, budget, kill switch and run receipt.

Agent	Назначение
Planner Agent	Decomposes task graph.
Research Agent	Gathers sources.
Architect Agent	Designs contracts.
Builder Agent	Writes implementation package.
Verifier Agent	Checks tests and acceptance gates.

Agent	Назначение
Archivist Agent	Creates receipts, docs, changelog and graph links.

- Every run records input sources, model, tool calls, data touched, tokens, cost, errors, artifacts and verification status.
- Agents cannot bypass permissions, budgets or receipts.
- Memory write requires policy and approval when sensitive.
- Every long-running agent has a stop condition and kill switch.

15. Workflow Runtime: трассируемые автоматизации

Workflows connect triggers and actions, but must be debuggable. A workflow creates visible artifacts and receipts, not hidden mutations.

Trigger → **Condition →** **Action →** **Artifact →** **Event →** **Receipt →** **Visible state**

- Source imported -> note draft -> knowledge candidate -> receipt.
- Book highlight created -> link to book -> suggest note -> graph update preview.
- Device offline -> smart-home alert -> action proposal.
- Meeting transcript imported -> summary -> task candidates -> user approval.
- Model budget near limit -> notify user -> pause non-critical agents.

16. Artifact Ledger and Data Control

Artifact Ledger is the trust spine. Data Control is the control tower. For every important result, LifeOS must answer what happened, where it came from, what data was touched, whether AI or cloud was used, what can be approved, rejected, exported, deleted or rolled back.

SourceImportReceipt **NoteDraftArtifact** **KnowledgeCandidateArtifact** **DecisionReceipt**
ApplyPreviewArtifact **RollbackPreviewArtifact** **BackupSnapshot** **AgentRunReport**
ModelCallReceipt **WorkflowRunReceipt** **ObsidianVaultScanReport** **SmartHomeAutomationReceipt**
ScreenContextReceipt **MediaTranscriptReceipt** **MarketplaceInstallReceipt** **QCReportArtifact**

17. UI Shell и пользовательские поверхности

Surface	Purpose
Home	High-level personal operating cockpit.
Life Feed	Stream of artifacts, events and channels.

Surface	Purpose
Search	Find anything.
Graph	See relationships.
Builder	Create and edit systems.
Artifacts	Inspect results and receipts.
Agents	Run and manage AI workers.
Model Hub	Manage providers, local models, BYOK and budgets.
Data Control	Consent, export, delete, rollback and provenance.
Market	Install systems, themes, renderers, agents, workflows and adapters.
Vault	Notes, files, sources and Obsidian-like space.
Command Palette	Universal command layer.
Settings / Trust Center	Permissions, secrets, security and health.

Главные способы взаимодействия: Feed показывает поток, Search находит, Graph объясняет связи, Chat даёт командный диалог, Builder создаёт системы, Artifacts дают доверие. Нельзя оставить только один из этих режимов.

18. Ключевые пользовательские сценарии

Первый запуск

Пользователь видит Capture, Feed, Search, Systems and Builder, а не 80 пунктов меню.

Импорт Obsidian

Scan vault, parse markdown/frontmatter/ backlinks, import attachments, build graph, index search, create ImportReceipt, offer rollback.

Создание системы

Builder creates SystemManifest, entities, fields, relations, views, actions, channel, renderers, permissions and receipts.

Настройка дизайна

User switches renderer, theme, density, grouping and layout without changing underlying data.

Локальная модель

User connects Ollama, LM Studio or custom endpoint; private tasks route local by default.

Jarvis

Permissioned screen context; local processing by default; suggestions become artifacts; actions require approval.

Smart home

Home Assistant-like adapter imports devices, rooms, scenes, events and safety policies.

Marketplace package

Install preview shows entities, views, agents, permissions, migrations and rollback before confirmation.

19. Subsystem families

Obsidian / Vault / PKM

One-way import first; preserve folder structure, frontmatter, tags, backlinks, embeds, attachments, timestamps, source paths, receipt and rollback.

Knowledge / Memory / Graph

Notes become source-backed knowledge only through reviewable promotion. Memory is policy-driven, not hidden profiling.

Search

Keyword, semantic, type, source, date, system, status, privacy and graph-neighborhood search with receipts.

Today / Planning / Focus

Action previews, evidence, confirmations, weekly recovery, no hidden life mutation.

Work / Projects / Meetings / Chat

Projects, meetings, chat threads, agent queues, source-backed summaries and no hidden external send.

Reader / Media / Courses

Books, highlights, reader notes, audio/video, transcripts, player progress, course paths and source citations.

Screen Companion / Jarvis

Permission gate, active window context, local OCR/vision, suggestion engine, action proposals and receipts.

Smart Home

Devices, rooms, scenes, sensors, automation rules, local adapters, safety tiers and degraded mode.

Marketplace / Creator

Systems, themes, renderers, layouts, agents, workflows, adapters, templates, install/update/rollback lifecycle.

Personal Twin / Wellbeing

Safe hypotheses, rhythm, recovery, anti-shame planning and user-verified memory; no diagnosis or hidden profiling.

20. Permissions, safety and degraded mode

Every subsystem, package, agent and adapter must request capabilities. High-risk capabilities require explicit confirmation, policy, receipt and often local-only defaults.

read_notes write_notes read_files write_files read_screen capture_screen control_mouse_keyboard read_smart_home control_smart_home call_cloud_model use_local_model read_api_keys create_agent install_package run_background_job access_microphone access_camera export_data delete_data write_calendar send_external_message publish_publicly charge_payment

High-risk action	Required behavior
Screen capture	Disabled by default; per-app allowlist; receipt.
Cloud call with private data	Explicit consent and model-call receipt.
Smart-home door/camera/security control	Strong confirmation, local-first, audit trail.
External send/share	Preview and confirmation.
Destructive delete	Preview, grace window or rollback path.
Payment/checkout	Provider configured, user confirmation, receipt.

Failure	Expected result
Search index failed	Notes and graph remain open; search shows stale warning.
Local LLM offline	Local-only tasks pause; cloud fallback only if allowed.
Obsidian import failed	Partial receipt; rollback available; source vault untouched.
Smart-home adapter offline	Smart-home degraded; LifeOS shell alive; controls paused.
Plugin failed	Package disabled or degraded; kernel remains alive.

21. Roadmap и порядок сборки

Первый milestone не должен быть reader demo. Сначала нужно доказать universal artifact stream, presentation runtime, adapter shell and system factory.

Milestone	Scope
M0	Universal Artifact Stream + Presentation Runtime + Adapter Shell
M1	Kernel skeleton: identity, permissions, module registry, lifecycle, events, artifacts, health.
M2	System Factory MVP: entities, fields, relations, views, actions, workflows, manifests.
M3	Data Fabric MVP: relational, document, object, event, artifact, search, graph, secrets.
M4	Notes, Search, Graph and Obsidian import.
M5	Model Fabric MVP: providers, BYOK, local endpoints, routing policies, receipts.
M6	Agent and Workflow MVP with kill switch and receipts.
M7	Marketplace and package shell.
M8	Smart-home adapter MVP.

Milestone	Scope
M9	Screen/Jarvis MVP.
M10	Reader, media, transcription and learning systems.

First proof

Create 3 artifact types, render each in Telegram/card/table/timeline modes, connect to search, graph, feed, Data Control, artifact inspector and health panel.

22. Дисциплина AI-разработки

LifeOS нельзя строить, просто бросив весь корпус в Fable или Cursor. Нужны маленькие implementation packages, source checks, tests, receipts and scope gates.

- Every package specifies goal, allowed files, forbidden files, route/API impact, smoke groups, acceptance gates and stop condition.
- Every AI change produces small diff, tests/checks run, artifact receipt, risk note and next package suggestion.
- No invented API, no invented repo file, no broad rewrite, no hidden cloud call, no destructive action, no false done.
- Find existing helper before writing a new helper.
- Use Scope Preservation Ledger when conflict appears.

23. Модель сценарного покрытия

A broad product cannot be protected by a flat function list. It needs scenario dimensions: product layer, object type, operation, risk, interface and execution state.

Dimension	Examples
Product layer	capture, source, note, knowledge, memory, graph, search, today, planning, work, chat, media, screen, smart-home, marketplace, builder, model, agent, data-control.
Object type	file, note, message, artifact, receipt, book, highlight, transcript, device, room, task, project, package, model, agent, workflow.
Operation	create, import, edit, approve, defer, reject, link, search, render, summarize, route, export, delete, rollback, install, uninstall, sync.
Risk	privacy, cloud leakage, hidden mutation, destructive action, false readiness, stale source, license issue, model cost, bad UX, degraded subsystem.
Interface	feed, card, table, graph, timeline, dashboard, reader, command palette, chat, data-control, settings, mobile, desktop.
Execution state	alive, degraded, failed, disabled, permission-blocked, provider-unavailable, index-stale, needs-config, owner-gated.

Completeness rule

A feature is covered if it can be represented as SystemManifest + Entities/Fields/Relations + Views/Renderers/Themes + Actions/Triggers/Workflows + Artifacts/Receipts + Events/Channels + Permissions/Policies/Budgets + Search/Graph + Adapter/Package lifecycle + Health/Export/Rollback.

24. Master checklist: nothing gets lost

Area	Questions
Product identity	Does it preserve LifeOS as Personal System Operating Platform, not notes/tasks/chat/dashboard?
System Factory	Does it use primitives, manifests, entities, views, actions, events and permissions instead of hardcoded clones?
Artifact/Event/Presentation	Does every meaningful result become artifact with source, receipt, status and multiple renderers?
Data/Privacy	Is data local-first where possible, cloud calls explicit, secrets isolated, export/delete/rollback visible?
Agents/Models	Does it support BYOK, local LLM, model routing, budgets, tool permissions, memory policy and kill switch?
Adapters	Does it use mature engines through adapters/sidecars/libraries with license checks and degraded mode?
UX	Can user search, graph, feed, chat, build, inspect receipts and customize design?
Safety	No hidden mutation, screen capture, smart-home control, external send, payment or false readiness.

25. Одностраничная версия для инвестора и разработчика

LifeOS is a local-first personal operating platform that lets people unify their data, knowledge, AI, daily planning, devices, automations and custom systems in one private workspace.

Unlike a normal productivity app, LifeOS is not built as a fixed list of features. It is built around a System Factory: users and creators define systems using reusable primitives such as entities, fields, relations, views, actions, workflows, agents, artifacts, permissions, packages and renderers. This allows LifeOS to support notes, Obsidian migration, graph knowledge, search, planning, reader/media workflows, smart-home dashboards, local AI, marketplace packages and future user-created systems without hardcoding each as a separate monolith.

Every meaningful output in LifeOS is an artifact with provenance. Artifacts produce events, events flow into channels, and channels can be rendered as message feeds, cards, tables, dashboards, timelines, graph nodes or custom marketplace renderers. Design is therefore user-configurable across the product.

LifeOS is private by default. It supports local storage, export, backup, home-server deployment, BYOK, local LLMs and explicit cloud permission gates. AI agents operate under permissions, budgets, model-routing policies and receipts. Screen/Jarvis and smart-home features are permissioned subsystems, not hidden surveillance or unsafe automation.

The first proof is not the full OS. The first proof is the universal artifact stream: create/import data, produce artifacts, show them in multiple renderers, connect them to search/graph/feed/data-control, and demonstrate that new systems can be created and installed without rewriting the kernel.

26. Что LifeOS может делать

- capture anything quickly;
- store personal data privately;
- import notes, vaults, files, books, media and transcripts;
- show all events in customizable feeds;
- search across life data;
- show graph relationships;
- turn sources into notes, knowledge, tasks, plans and actions;
- ask AI with receipts and source trails;
- connect cloud, BYOK and local models;
- run controlled agents;
- build custom systems;
- install systems, themes, renderers, workflows and adapters;
- connect smart home;
- use permissioned screen-aware assistant;
- read books and extract highlights;
- transcribe media;
- plan day, week and focus sessions;
- manage work, projects, relationships, finance, travel, inventory and health-like context;
- control data, permissions, exports, deletes, rollbacks and backups;
- migrate from Obsidian and export back;
- run locally on laptop, desktop or home server;
- extend the platform without waiting for the core developer to manually add every future feature.

27. Что собирать первым в коде

Package / app area	Purpose
packages/kernel	Core lifecycle and contracts.
packages/artifacts	Artifact and receipt schemas.
packages/events	Event bus and event types.
packages/channels	Channel/stream runtime.
packages/presentation	Renderers, themes, layouts, view presets.
packages/system-factory	Primitives, system manifests, builder logic.
packages/permissions	Capabilities, policies, high-risk gates.
packages/data-fabric	Stores, indexes, secrets and backup abstractions.
packages/search	Full-text and semantic search adapters.
packages/graph	Relations and graph queries.
packages/model-fabric	Provider registry, BYOK, local endpoints, routing.
packages/agents	Agent manifest, runs, tools, budget, receipts.
packages/adapters	External engine adapters and health checks.
packages/packages-runtime	Install, update, rollback, compatibility and migrations.
apps/web/app/feed	Universal artifact stream.
apps/web/app/builder	System and presentation builder.
apps/web/app/artifacts	Artifact inspector.
apps/web/app/data-control	Trust and consent control tower.
apps/web/app/model-hub	Provider and model routing UI.
apps/web/app/systems	Installed and user-created systems.

Приложение A. Полный список visible routes

1. /alpha
2. /app/activities
3. /app/aliveness
4. /app/alpha
5. /app/billing
6. /app/books
7. /app/builder
8. /app/calendar
9. /app/capture
10. /app/chat
11. /app/community
12. /app/courses
13. /app/creator
14. /app/data-control
15. /app/design-studio
16. /app/domains
17. /app/entity-resolution
18. /app/events
19. /app/experiments
20. /app/finance
21. /app/focus
22. /app/graph
23. /app/health
24. /app/home
25. /app/import
26. /app/inbox/[sourceId]
27. /app/inbox/browse

28. /app/inbox/completed
29. /app/inbox/duplicates
30. /app/inbox/later
31. /app/inbox/ops
32. /app/inbox
33. /app/inbox/processing
34. /app/inbox/review
35. /app/inventory
36. /app/journeys/[journeyId]
37. /app/journeys
38. /app/knowledge
39. /app/learning
40. /app/life-admin
41. /app/market
42. /app/matching
43. /app/media
44. /app/memory
45. /app/model-hub
46. /app/notes
47. /app/notifications
48. /app/nutrition
49. /app/packages
50. /app
51. /app/planning
52. /app/player
53. /app/progress
54. /app/projects
55. /app/prompts
56. /app/qc
57. /app/relationships
58. /app/rhythm
59. /app/rules
60. /app/screen-companion
61. /app/search
62. /app/settings
63. /app/social
64. /app/start
65. /app/states
66. /app/today
67. /app/travel
68. /app/twin
69. /app/weekly
70. /app/work/availability
71. /app/work/meetings/[meetingId]
72. /app/work
73. /app/workflows
74. /capture
75. /entry/[painKey]
76. /entry
77. /
78. /pricing/checkout
79. /pricing
80. /roles/[roleKey]
81. /roles
82. /share

Приложение B. Полный список API routes

1. /api/adaptive-home/runtime
2. /api/ai-enhancement/runtime
3. /api/alpha/runtime
4. /api/billing/callback/[provider]
5. /api/billing/webhook/[provider]
6. /api/builder/runtime
7. /api/capture/quick
8. /api/chat/runtime
9. /api/commercial/runtime
10. /api/community/runtime
11. /api/courses/runtime
12. /api/creator/runtime
13. /api/data-control/runtime
14. /api/design-studio/runtime
15. /api/domains/runtime
16. /api/entity-resolution/runtime
17. /api/entry/runtime
18. /api/events/runtime
19. /api/focus/runtime
20. /api/health/runtime
21. /api/inbox/sources
22. /api/inventory/runtime
23. /api/knowledge/runtime
24. /api/lifegraph/runtime
25. /api/local-search/runtime
26. /api/marketplace/runtime
27. /api/matching/runtime
28. /api/media-transcript/runtime
29. /api/media/runtime
30. /api/memory/runtime
31. /api/model-hub/runtime
32. /api/notes/runtime
33. /api/notifications/[notificationId]
34. /api/nutrition/runtime
35. /api/pack-economy/runtime
36. /api/personal-twin/runtime

37. `/api/planning/runtime`
38. `/api/privacy-consent/runtime`
39. `/api/progress/runtime`
40. `/api/protocol/runtime`
41. `/api/qc/runtime`
42. `/api/recovery/runtime`
43. `/api/release-readiness/runtime`
44. `/api/rhythm/runtime`
45. `/api/screen-companion/runtime`
46. `/api/shortcuts/runtime`
47. `/api/social/runtime`
48. `/api/source-sentinel/runtime`
49. `/api/today/runtime`
50. `/api/travel/runtime`
51. `/api/trust-center/runtime`
52. `/api/v33/apply-preview/runtime`
53. `/api/v33/review-decision/runtime`
54. `/api/weekly/runtime`
55. `/api/work/runtime`
56. `/api/workflow/runtime`

Приложение C. 100 категорий улучшений

1. C01 Codex session stability
2. C02 VPN/Happ transport
3. C03 Prompt discipline
4. C04 Context capsule
5. C05 Git recovery
6. C06 Lockfile/npm reliability
7. C07 Preview/port discipline
8. C08 Package budget
9. C09 Local acceleration scripts
10. C10 Route impact map
11. C11 Deferred compute
12. C12 Metrics burn-down
13. C13 Evidence manifest
14. C14 State sync
15. C15 Browser smoke policy
16. C16 i18n RU-first
17. C17 Copy/mojibake quality
18. C18 Dangerous-action scan
19. C19 TypeScript/ESLint speed
20. C20 Next build/runtime
21. C21 Worktree/preview lane
22. C22 Data Control spine
23. C23 Flow Ledger
24. C24 Decision receipts
25. C25 Consent graph
26. C26 Privacy boundaries
27. C27 Encryption/key lifecycle
28. C28 Local-first AI
29. C29 Model Hub
30. C30 Source import
31. C31 Knowledge persistence
32. C32 Local Search
33. C33 LifeGraph
34. C34 Today/Planning/Focus
35. C35 First-use alpha loop
36. C36 Command Palette
37. C37 HTTP cache
38. C38 Builder systems
39. C39 Workflow Studio
40. C40 Design Studio
41. C41 Screen companion
42. C42 Media transcript
43. C43 Entity Resolution
44. C44 Marketplace/Pack economy
45. C45 Auth/passkeys
46. C46 Onboarding
47. C47 Product metrics
48. C48 Readiness ladder
49. C49 Branch/merge strategy
50. C50 Release focus
51. C51 QC deferred boundary
52. C52 No false completion

53. C53 Stale prompt detector
54. C54 Resume capsule
55. C55 Live run state
56. C56 GitHub verification
57. C57 Cloud Codex isolation
58. C58 Cloud environment setup
59. C59 OS process locks
60. C60 PB/runtime dependencies
61. C61 Visual clarity
62. C62 Accessibility
63. C63 Empty/error states
64. C64 User trust language
65. C65 Data map
66. C66 Transparency digest
67. C67 Cost transparency
68. C68 Automation safety
69. C69 Rollback/undo
70. C70 Anti-shame recovery
71. C71 Predictive Today
72. C72 Feature store
73. C73 Agent reflection
74. C74 Provider neutrality
75. C75 Device budget
76. C76 Offline/edge
77. C77 Data portability
78. C78 Source strength
79. C79 Claims/provenance
80. C80 Notes/Knowledge UX
81. C81 Search UX
82. C82 Graph UX
83. C83 Today UX
84. C84 Planning UX
85. C85 Focus UX
86. C86 Inbox UX
87. C87 Universal object contract
88. C88 Template/pack manifest
89. C89 Workflow debug
90. C90 Performance
91. C91 Security baseline
92. C92 Auth/access later
93. C93 Analytics local-first
94. C94 Monetization readiness
95. C95 QA maturity
96. C96 Migration hygiene
97. C97 Documentation minimalism
98. C98 Codex output discipline
99. C99 Owner decision gates
100. C100 Full canon roadmap

Приложение D. 22 tactical axes

cache **continuation** **cost_guard** **evidence** **fallback** **fast_check** **full_check** **input_contract** **metric** **mobile** **no_duplicate** **owner_gate** **receipt_path** **rollback** **runtime_hook** **safety_boundary** **scope_gate** **stale_guard** **state_sync** **test_fixture** **ui_marker** **ux_copy**

Финальная самопроверка

This PDF deliberately preserves current v33 modules, visible routes, API routes, improvement categories, tactical axes, System Factory, artifact/event/channel/presentation runtime, user-controlled design, adapter strategy, Obsidian/vault/PKM, graph/search/memory, data fabric, privacy/local-first/home-server, BYOK/local LLM/model router, agents/workflows, marketplace/package economy, screen/Jarvis, smart-home, reader/media/transcription/courses, daily planning/

focus/work, personal twin/wellbeing/recovery, real-life domains, permissions/safety, failure isolation, execution discipline, first milestone and anti-loss checklist.

> **Actual completeness mechanism**
>
> If a future feature is not named, it is still covered if it can be represented as System = Entities + Fields + Relations + Views + Actions + Triggers + Workflows + Agents + Artifacts + Receipts + Permissions + Renderers + Packages + Adapters + Policies + Health.